

ARC-AGI-3 Part 1: Formalization and Survey of Methods (2nd ed.)

Bo Hou · 2026-10-04 · Series: Deriving World Models for ARC-AGI-3 · Second edition

Part 1 of the series: formalize the problem, fix one notation, rewrite the existing methods in it, and locate the gaps. This second edition went through a section-by-section review; Section 11 records the key corrections.

0 In Plain Words: What the Task Tests

ARC-AGI-3 is a set of small games with no manual: the screen is a 64×64 grid of colored cells, the agent can only press keys or click cells, and both rules and goals must be discovered.

- **What it sees:** one 64×64 frame per step, each cell one of 16 colors; energy bars and indicator boxes are drawn inside it. It also knows the current level, whether the game ended, and which actions are available.
- **What it can do:** at most 7 actions plus reset — up/down/left/right, an “interact” key, clicking a cell, undo. The same key means different things in different games, and each game exposes only some of them.
- **What it must do:** each game has several levels; later levels are harder and weigh more. The win condition is never stated; it has to be found by trying.
- **How it is scored:** against the action count of an ordinary first-time human. Using twice as many actions earns $1/4$ of the level score; an uncleared level earns 0. Random trying is expensive, and so is exploration.
- **Evaluation:** Kaggle, one GPU, no internet, at most 9 hours per submission. The 25 public games can be played locally; evaluation uses 110 private games whose mechanics are deliberately different.

In one sentence: it does not test whether you can play a given game, but whether you can learn an unseen game with very few attempts.

1 Problem Formalization

Each game is an unknown deterministic Moore machine: the internal state z determines the frame x , and the agent only sees x . Observation fields were checked against the arc-agi 0.9.9 / arcengine source.

1.1 Notation

Symbol	Meaning
$g \in G$	A game; $G_{pub} = 25$ public games, $G_{priv} = 110$ private games
$\ell = 1, \dots, L_g$	Level index; L_g = number of levels (win_levels)
t	Real action step
$x_t \in \{0, \dots, 15\}^{64 \times 64}$	Frame after step t (a step may return several frames; the last one is taken)
σ_t	Game state: NOT_FINISHED / WIN / GAME_OVER
c_t	Levels completed
$A_t \subseteq \{0, \dots, 7\}$	Available actions: 0 RESET, 1–4 directional, 5 interact, 6 click, 7 undo

Symbol	Meaning
a_t	Action; $a_t = 6$ carries coordinates $(u, v) \in [0, 63]^2$
$o_t = (x_t, \sigma_t, c_t, L_g, A_t)$	Full observation
z_t	True internal state
T^*, R^*	True transition and rendering
$D_t = \{(x_i, a_i, x_{i+1})\}_{i < t}$	Interaction record
ϕ, s	Object abstraction and object-level state (Section 1.3)
$h = (Enc_h, T_h, R_h, G_h)$	A world-model hypothesis (Section 3.2)
$\Lambda = (\theta, \text{harness})$	Outer prior: model weights + prompts/tools/control flow
$b_{g,\ell}, n_{g,\ell}$	Human baseline and AI action counts for level ℓ

1.2 Dynamics: z, T^*, R^*

$$z_{t+1} = T^*(z_t, a_t), \quad x_t = R^*(z_t), \quad a_t \in A_t$$

- **z**: the current values of all member variables of the game class (object positions, energy, switches, level, timers...).
- **R***: draws z as a frame. What is drawn (energy bar, objects) is visible; what is not (motion direction, cooldown timers, undisplayed switches) is hidden state. Analogy: z is the class's private members, x its outward presentation.
- **T***: deterministic rules. Output depends only on state (a Moore machine); the frame is a projection of z , not z itself. Tycho calls this a parameterized rendered deterministic Moore machine.
- If the same (x, a) yields several different x' , there is off-screen state and history is needed to infer it.
- Priors: Core Knowledge only (objectness, geometry and topology, agentness; secondary sources add basic physics); no digits or letters; quantities appear visually as lengths or counts.

1.3 Objects and Object State

The object-level state S is a tree: objects may contain sub-objects; each object is $O = (\kappa, \tau, \nu)$.

- **κ identity**: tracks the same object across frames.
- **τ type**: determined by color and shape mask, independent of position.
- **ν attributes**: position = top-left corner ($\min r, \min c$) of the minimal axis-aligned rectangle containing the object's pixel set P , size $h \times w$; shape = binary mask $M \in \{0, 1\}^{h \times w}$ in the bounding box; appearance = color map $C \in \{0, \dots, 15\}^{h \times w}$ (non-object cells transparent); layer d (occlusion order). (position, M, C) redraws the object exactly.
- **Occlusion**: P is the object's full extent (occluded parts completed by cross-frame tracking), otherwise the position jumps; hence $S_t = \phi_h(x_{t-k}, \dots, x_t)$ uses history — Core Knowledge object permanence.
- **ϕ_0 vs ϕ_h** : same-color 4-connected component segmentation ϕ_0 is the default prior; the ϕ_h actually used belongs to hypothesis h and can be corrected by data (e.g., merging blocks that always move together).
- **Roles are not in ϕ** : “the one I control, the one that refills energy, the goal” is learned only through interaction and belongs to T_h and G_h .

2 Scoring and Global Objective

The goal is to clear as many and as late levels as possible with as few real actions as possible; actions beyond the human baseline are penalized quadratically. Per-level score (0 if not cleared):

$$r_{g,\ell} = \min(1.15, (\frac{b_{g,\ell}}{n_{g,\ell}})^2)$$

Per-game score (weighted by level index) and total score:

$$S_g = \frac{\sum_{\ell=1}^{L_g} \ell r_{g,\ell}}{\sum_{\ell=1}^{L_g} \ell}, \quad S = \frac{1}{|G|} \sum_{g \in G} S_g$$

- $b_{g,\ell}$ is the upper median of first-time human players' action counts.
- **How $n_{g,\ell}$ is counted** (arc-agi 0.9.9 scorecard.py): each ACTION1–7 adds 1; an ordinary RESET adds one reset and one action; a full reset (full_reset) opens a new play record whose action count restarts at 0. The source caps a level at 115 (out of 100), matching 1.15. Whether Kaggle's server uses the same code could not be verified directly.
- The 1.15 cap means about 7% fewer actions than humans already maxes out ($\sqrt{1.15} \approx 1.07$).
- Example: human 22 steps, AI 44 steps $\rightarrow$ 0.25. A mean of 1.6% cannot be read as “8× the actions on every level”, because uncleared levels count as 0.

Global objective: a policy π (modeling plus planning) maximizing the expected private-set score:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{g \sim G_{priv}} [S_g(\pi)]$$

3 Two-Layer Modeling Framework

The outer layer defines “the language in which game programs are written”; the inner layer writes the current game's program in that language on the spot and uses it to clear levels. Together they form a meta-learning problem.

3.1 $\Lambda, \mathcal{H}, h$: Thinking in Base Classes

- **Λ (outer)**: a library of base classes and interaction primitives — “movable object”, “wall”, “counter bar”, “collision triggers an effect”, “pass only if a condition holds”. In real systems $\Lambda = (\theta, \text{harness})$: θ are the LLM weights, the harness is prompts, tools and control flow.
- **$\mathcal{H}$ (hypothesis space)**: all game programs writable with Λ . A richer Λ gives a larger $\mathcal{H}$ — more expressible games, harder search.
- **h (inner)**: one concrete program in $\mathcal{H}$: which objects exist and their classes, what each action acts on, what happens on contact, what counts as winning.
- **Explicit goal of outer learning**: learn to “quickly and concisely generate a program that generates this game”. What is learned is not new rules but the ability to learn new rules; private-set mechanics are deliberately out of distribution, so Λ should store low-level primitives rather than concrete rules.

3.2 Inner Layer: Inferring Hidden State, Transition, Rendering

T^* is unobservable because Z is latent; h maintains its own estimate $\hat{Z}$ inferred from history, renders it back to a frame and compares with the observation:

$$\hat{z}_t = \text{Enc}_h(x_{\leq t}, a_{< t}), \quad \hat{z}_{t+1} = T_h(\hat{z}_t, a_t), \quad \hat{x}_{t+1} = R_h(\hat{z}_{t+1})$$

- $\hat{Z} = X$ only when everything is drawn. “Inferer + transition + renderer” mirrors the “encoder + transition + decoder” of variational world models, except that all three are programs here.
- T_h approximates T^* in the observable sense, and only on the rules that matter for clearing levels.

The per-step loop (actions always run in the real environment):

1. Observe x_t ; append (x_{t-1}, a_{t-1}, x_t) to D_t .
2. Verify and revise h so it agrees with D_t .
3. If h is trustworthy $\rightarrow$ search T_h for a sequence reaching G_h ; otherwise $\rightarrow$ take the action on which candidate hypotheses disagree most.
4. Compare predictions with real frames while executing; on mismatch return to step 2.

Trust criterion: all candidates h' consistent with D_t and with $C(h') \leq C(h) + \epsilon$ make identical predictions along the planned path. When this fails, the most disputed step is exactly the action to explore in step 3. ϵ is open, and whether the criterion fully closes will be settled in practice.

3.3 Modeling Cost

Direction: learn the rules that matter with the fewest probes. The computable version replaces the unobservable $d(T^*, T_h)$ by data fit on D_t plus a complexity term (δ is the Kronecker delta):

$$L_{fit}(h; D_t) = \sum_i (1 - \delta(R_h(T_h(\hat{z}_i, a_i)), x_{i+1}))$$

$$h_t = \arg \min_{h \in H} C(h) + \lambda L_{fit}(h; D_t)$$

- L_{fit} : number of prediction errors on recorded steps; exact replay verification is the hard constraint $L_{fit} = 0$.
- $C(h)$ **model complexity**: ideally Kolmogorov complexity $K(h)$ (uncomputable), approximated by a normalized description length (not character count — long variable names are not complexity); “degrees of freedom” applies only to parametric models.
- Formally similar to MDL (minimum description length), used only as a reference.
- The solver is discrete search over program space (an LLM proposes edits, verifiers filter), not gradient descent.

3.4 Play Cost and Alternating Optimization

$$\min n_{g,\ell} = n_{g,\ell}^{explore} + n_{g,\ell}^{play}$$

- Exploration and planning alternate under one global objective; exploration is valued by how much it reduces n^{play} .
- When to pay for modeling and when to act directly: Tycho calls this active abstraction.

3.5 Outer Layer: Meta-Learning and Privileged Information

$$\omega^* = \arg \min_{\omega} E_{T \sim p(T)} [L_T(f_{\omega}(D_T^{train}); D_T^{test})]$$

f_{ω} adapts on a task's D^{train} and is evaluated on D^{test} ; the outer loop optimizes ω . Here $\omega = \Lambda$, a task is a game, D^{train} is earlier levels and D^{test} later levels:

$$\Lambda^* = \arg \max_{\Lambda} E_{g \sim G_{train}} [S_g(\pi_{f_{\Lambda}})]$$

Layer	What is learned	When	Analogy
Outer Λ	Base classes/interaction primitives, modeling method, model weights	Offline; during evaluation too if games run sequentially and rules allow	Physical fitness, nerves and muscles

Layer	What is learned	When	Analogy
Inner h	This game's $\text{Enc}_h, T_h, R_h, G_h$	At evaluation, on the spot	One specific routine

- **Privileged information:** the 25 public games' source code is available only in training; learning succeeds when outer + inner together yield a program semantically equal to the real code. At inference only L_{fit} and next-frame accuracy remain as proxies. This is teacher–student distillation, unlike ordinary RL where training and inference see the same signals.
- **Contrast with robotics:** sim-to-real deployment also checks an L_{fit} -like consistency; online adaptation (e.g., RMA, from memory) tunes parameters only. ARC must revise rule structure online.
- **Contrast with Lux AI S3:** Lux rules are public, so the inner layer only infers parameters (learning parameters); ARC's inner layer infers structure (learning structure).

4 Taxonomy and Class A: No LLM

Methods are grouped by the form of h ; offline vs online is an attribute. Higher scores sit closer to class C, and no class-C method exists offline yet.

Class	Form of h	L_fit verification	Methods
A No LLM	q_θ (change or not), state graph + value model	None	StochasticGoose, Blind Squirrel, Graph-Based Exploration
B LLM + textual h	Natural-language rule hypotheses, not executable	None	Duck, Reki, forge, AERA (LLM version), Symbolica
C LLM + executable h	Python programs	Yes (replay)	Rodionov EWM, Tycho, OPINE-World

4.1 StochasticGoose (Preview 1st, 12.58%)

The inner layer learns only whether the frame changes, not T_h or G_h :

$$q_\theta(x, a) \approx P(\text{frame changes} \mid x, a), \quad \pi(a \mid x) \propto q_\theta(x, a)$$

- q_θ is a CNN trained online: one head for ACTION1–5 and one 64×64 click head — the value at (u, v) is the probability that clicking that coordinate changes the whole frame, not that this pixel changes.
- Labels are change/no-change; a buffer of up to 200k entries deduplicated by (x, a) hash; **the buffer is cleared and the model reset on every new level** — no outer layer, not even cross-level transfer.
- The objective itself is off: change $\neq$ win; one experiment shows that under current scoring even a perfect q_θ gains almost nothing.

4.2 Blind Squirrel (Preview 2nd, 6.71%)

- State graph: nodes are hashed frames, edges (x, a, x') ; self-loops and cycle-inducing actions are pruned, and actions known to cause GAME_OVER are avoided. A lookup-table record of rules that does not generalize.
- Value model: after a score increase, shortest steps-to-win are back-labeled on the graph (BFS) and used to train $V_\psi(x, a)$ — an **ImageNet-pretrained ResNet-18** with custom stem and head, taking the 64×64 grid plus an action embedding; fine-tuned on earlier levels of the same game and unused before level 1 ends. Its scores combine with a “valid actions model” to pick actions.

- In two-layer terms: outer = generic ImageNet visual features; inner = cross-level fine-tuning within the game. It learns value (distance to winning), not how the game works; its labels target the goal, better by design than 4.1, though the Preview result is reversed (only 3 games).
- In essence: systematic exploration + dynamic-programming labels + network fitting, iterable (expert iteration).

4.3 Graph-Based Exploration (Preview 3rd)

Training-free: segments frames into components, ranks actions by visual salience, keeps a state graph, and goes to the nearest untested (state, action); status bars are masked to cut the state count. Solves a median of 30 of 52 levels across 6 games.

5 Class B: LLM + Textual h

Same broad direction: outer $\Lambda = (\theta, \text{harness})$, inner in-context learning, h a textual hypothesis; the detail that differs is how actions are produced (computed by a program vs chosen directly).

5.1 Duck (Milestone 1 1st; semi-private 1.21%, public 1.60 ± 0.45)

Source: duck-harness prompts.py and configuration files.

- **State:** X_t and views of it; the raw numeric grid is withheld. `current_frame` exposes only `.ascii` (letter-coded grid, read small regions only), `.segmentation`, `.step`, `.level`, `.shape`; each turn also attaches a grid image.
`.segmentation = \phi_0(X_t)`: each node is a 4-connected same-color object with id, color, hash (position-independent color+shape signature for cross-frame tracking), pixels, boundary (outer contour corners), children (enclosed objects), plus an `adjacency_list` — nearly identical to the object definition in Section 1.3.
- **Interaction:** `history`, `transitions` (action, before/after frames, result) = D_t ; `last_action_result` (frame changed, level completed, GAME_OVER); `valid_actions = A_t`; `action([...])` executes real actions in batches from Python.
- **What the code does:** frame diffs, object tracking, checking whether only the HUD changed, discriminating probes when unsure, and BFS/pathfinding for shortest sequences once confident.
- **What h is:** a textual “working world model” in the conversation (objects, action effects, goal, uncertainties, plan). Every Python call starts fresh and code is not kept; the context is 32K tokens and the oldest messages are evicted, so h is lost with them.
- **Two layers:** θ = off-the-shelf Qwen3.6-27B-FP8, no fine-tuning in the config; harness = system prompt (priors: scenes consist of objects, edge strips are usually timers, an avatar may not exist; procedure: maintain a world model, probe then search) + tools. θ comes from pretraining; the prompt was revised by people who studied failures on public games (human-in-the-loop offline outer learning).

$$h_t, a_t = \text{LLM}(\theta, P, D_t)$$

- The official claim that Duck is StochasticGoose’s spiritual successor refers to lineage (both Tufa Labs, Smit involved), not method.

5.2 Reki (Milestone 1 2nd, score not published)

Vision-LLM as policy: each turn renders recent frames as labeled images for a local Gemma-4-31B and asks for JSON — what changed, a short plan, the next 1–4 actions; reflection memory refreshes about every 10 steps. Built on the official GPT-OSS-120B template, with JSON self-repair, legal-action constraints, a plan queue, a hand-written click heuristic

(small, rare-colored, button-like shapes first) and “dead signatures” (object types whose clicks never change anything are skipped for the level); every feature toggles via environment variables for ablation. It differs from Duck not by multimodality (Duck also uses images) but by writing no code, doing no search and choosing actions directly.

5.3 forge (Milestone 1 3rd, score not published)

Same flow as Reki, packaged as a profile-driven framework: multi-candidate action generation + an arbiter, an optional confidence prompt (forcing safe, reversible moves when unsure); the top run switched these off. The author notes local public-game checks are not a reliable leaderboard proxy.

5.4 AERA (paper arXiv 2605.25931)

- **LLM agent:** Qwen2.5-0.5B outputs HYPOTHESIS / UNCERTAIN / NEXT_ACTION / REASON each step; three explicit phases EXPLORE (budget $\max(5, \min(30, [0.4 \cdot \text{level}-1 \text{ baseline}]))$) → VERIFY (1–3 falsification steps) → PLAN (plan/confidence/fallback, back to EXPLORE on surprise); the length of the UNCERTAIN field proxies belief entropy for switching, which the author calls a weak heuristic; memory keeps the last 10 steps. Self-reported 0.2116 on the 25 public games (4 solved).
- **The private 0.30 is not this agent:** the competition entry is a BFS pre-solve at episode start (180 s per level) with a heuristic fallback — no learning, closer to class A.
- **Main contribution is a benchmark critique:** all 25 public games are reachable by non-intelligent strategies, and a null-coordinate bug bypasses 18 games in one step. Its RHAE is not level-weighted, so its numbers are not comparable with the official ones.

5.5 Symbolica Arcgentica (frontier model, 36.08 on the public set on launch day)

Recursive agent orchestration that interacts with the game through code; h implicit, no explicit modeling cost.

6 Class C: LLM + Executable h

The outer layer is again LLM + harness; the difference is that h is a runnable program verified by L_{fit} . All results so far use frontier API models on the public set without held-out evaluation.

6.1 Rodionov EWM (GPT-5.5: 58.12, 15 games fully solved; GPT-5.4: 41.29, 8 games)

- h : a Python codebase with fixed interfaces (state representation, transition prediction, goal test, planner).
- Modeling cost: verifiers after every edit require $L_{fit} = 0$ (exact replay); refactoring lowers $C(h)$, described by the author as a proxy for an MDL simplicity bias.
- Plans in T_h , compares step by step during execution, stops to revise on mismatch. Each playthrough starts from a clean workspace, so the outer layer never accumulates; unintended information channels were audited (agents actively read game source and human baselines).
- Follow-up ablation: ewma_sv (executable + simplification + replay verification) vs twma (purely textual h).

6.2 Tycho (Opus 5 / GPT-5.6 Sol: 100.00; Opus 4.8: 88.49, no-model baseline 79.07)

- Formalized as a parameterized rendered deterministic Moore machine; actionable frames are separated from animation, level-completion and game-over frames.

- h : a Python simulator with freely chosen state variables, initialization and transition rules, a renderer and an outcome classifier; latent variables are allowed so one frame need not determine the next (isomorphic to Enc_h, T_h, R_h in Section 3.2). On ls20 the generated State held avatar position, remaining energy, carried shape and color, refills, launchers, moving-object directions and fog-hidden terrain.
- The key variable is when to build and when to use the model: “the actor requests the builder” works best; automatically repaired models are more accurate yet score lower. About \$2.99k per Opus 5 run.

6.3 OPINE-World (Opus 4.8: 78.40, same-model comparison in the Tycho paper)

Parses X_t into an object set $S_t = \{o^i\}$ with $o^i = (\kappa^i, \tau^i, v^i)$; pairs objects across frames, computes effect signatures and fills effect-table rows (τ^i, a_t, u^i) ; Dirichlet row diagnostics, type uncertainty and “ontology error” decide whether the abstraction suffices; only once the ontology is stable does it synthesize an executable T_h , verified by full next-state equality.

6.4 Unclassified (form of h not verified)

- **Kepler** (Opus 5: 100.00): “auditable world models”, about \$778 per run, 36% fewer tokens than Tycho.
- **ARA** (AgentNativeResearchLab): builds a structured h during play, with a prediction ledger carrying confidences; score not found.

7 Why Online Methods Win: Outer θ , Inference Budget, Inner Representation

The evidence points to: the gap comes mainly from outer model capability θ , then inference budget; the inner learning mechanism is the same on both sides, and an explicit T_h with verification adds about ten points on frontier models.

Factor	Evidence	Conclusion
Outer θ (model weights)	Same harness, different model: EWM GPT-5.5 58.12 vs GPT-5.4 41.29; Tycho Opus 4.8 88.49 vs Opus 5 / GPT-5.6 Sol 100. Duck on GPT-5.4 solves a similar set of games as EWM at an order of magnitude lower cost	Whether a game is solvable depends on θ ; cost depends on the harness
Inference budget	Tycho + Opus 5 about \$2.99k and 1.3B tokens; Kepler about 0.86B tokens; Kaggle one GPU, 9 hours; Duck context 32K	“How” the inner layer learns is the same; “how long” differs by orders of magnitude
Inner representation (explicit T_h + verification)	Tycho no-model baseline 79.07; best world-model policy +9.42	Contributes, but not the main cause

- **Harness**: the program around the LLM — how observations are presented, which tools exist, how prompts are written, how the loop and context are managed; in this framework, the part of Λ other than θ .
- **Model scale**: OpenAI has not disclosed GPT-5.4’s size, but external estimates are all in the trillions — about two orders of magnitude above 27B. Capacity need not differ 100×, but the difference is qualitative.
- **Confound**: GPT-5.6-sol and similar models were released after the public games and were not tested on held-out games; part of the public-set scores may come from having seen these games.

8 Background Methods

Grouped by inner vs outer layer; entries marked “memory” were not individually re-checked.

Method	Layer	What it learns	Relation to this problem	Source
EMPA (Tsividis et al.)	Inner	Bayesian inference over VGDL rule space $\mathcal{H}$ + exploration + planning	Inspiration for ARC’s efficiency scoring; VGDL expresses only simple collision interactions	Cited by ARC
TheoryCoder	Inner	LLM synthesizes code for T_h for bilevel planning	Goes beyond VGDL	TMLR 2025
WorldCoder (Tang, Key, Ellis)	Inner	Builds T_h by writing code and interacting	Early programmatic world model	2024
Code World Models + MCTS (Dainese et al.)	Inner	LLM generates code T_h guided by MCTS	Same line	2024
PoE-World	Inner	T_h = product of programmatic experts	Compositional rules	2025
DreamCoder	Outer	Learns a reusable program primitive library Λ from many tasks	Closest paradigm to “primitive library as prior”	memory
Voyager	Outer	Continually growing agent skill library	Cross-task skill reuse	memory
Dreamer / MuZero	Inner	Neural T_h + planning/search in imagination	Needs heavy interaction; unsuited to few-shot first contact	memory
MAML / RL ²	Outer	ω = initialization / weights of an adapting RNN	Standard meta-learning (Section 3.5)	memory
AlphaGo / AlphaZero	Single task	SL + RL + MCTS under known rules	Not meta-learning; search → distillation is reusable	memory
RMA	Inner (parameters)	Online estimation of environment parameters	Tunes parameters, not structure — contrast with ARC	memory

9 Comparison Table

Ordered by class. Online scores are public-set RHAЕ (out of 100); offline, Preview and AERA self-reported numbers use different protocols and are not directly comparable.

Method	Class	Setting	Score	Outer Λ	Inner h	Modeling cost	Play cost
Tycho	C	Online, Opus 5 / GPT-5.6 Sol	Public 100.00	Fixed framework + LLM	Executable simulator (with latents)	L_{fit} verification	Plan in model; actor decides when to build/use
Tycho	C	Online, Opus 4.8	Public 88.49	Same	Same	Same	Same
OPINE-World	C	Online, Opus 4.8	Public 78.40	Object parsing + effect table	T_h synthesized after ontology stabilizes	Ontology error; next-state equality	Not read

Method	Class	Setting	Score	Outer Λ	Inner h	Modeling cost	Play cost
Rodionov EWM	C	Online, GPT-5.5	Public 58.12	Fixed interfaces + verifier + refactoring	Python codebase	$L_{fit} = 0 + \text{lower } C(h)$	Plan in model, compare on execution
Symbolica	B	Online, frontier model	Public 36.08	LLM + harness	Implicit	None explicit	None explicit
StochasticGoose	A	Preview	12.58%	None	$q_\theta(x, a)$, reset per level	Change cross-entropy	$\pi \propto q_\theta$
Blind Squirrel	A	Preview	6.71%	ImageNet ResNet-18	State graph + V_ψ , cross-level fine-tuning	None (exact record)	Value regression
Graph-Based Exploration	A	Preview	3rd, median 30/52 levels	None	State graph	None	Nearest untested action
Duck	B	Kaggle offline	Semi-private 1.21%	Qwen3.6-27B + prompt + tools	Textual world model in context	None explicit	LLM writes BFS etc.
Reki	B	Kaggle offline	Not published	Gemma-4-31B + click prior	Reflection memory + dead set	None explicit	LLM picks 1–4 steps directly
forge	B	Kaggle offline	Not published	Same as Reki	Same as Reki	None explicit	LLM picks directly
AERA (LLM)	B	Paper	Public 0.2116 (own protocol)	Qwen2.5-0.5B + three-phase flow	HYPOTHESIS field	UNCERTAIN length as entropy proxy	PLAN phase outputs a plan
Kepler	Unclassified	Online, Opus 5	Public 100.00	Auditable WM framework + LLM	Not read	Not read	Not read
ARA	Unclassified	Online, Opus 4.8 etc.	Not found	LLM	Structured h	Prediction ledger	Not read

10 Gaps and Open Questions

Three gaps: an explicit outer Λ (a primitive library learned offline and updatable during evaluation); class-C methods under offline local-model conditions; making “how to learn” explicit (which action to explore, how to weigh hypotheses).

- Milestone 1 2nd/3rd place scores; Milestone 2 (Sept 30 deadline) results
- Form of h and cost definitions for Kepler and ARA
- Whether Kaggle runs the 110 games sequentially in one process; whether rules allow accumulating Λ across games during evaluation
- Whether rules allow training local models on API-generated data
- Whether Kaggle’s server scoring matches the SDK `scorecard.py` (RESET counting)
- The value of ϵ in the trust criterion; whether the criterion closes or needs a proof
- A concrete measure for $C(h)$ (definition of normalized description length)

11 Derivation Log: Key Corrections During Review

This section records the back-and-forth from the first edition to this one: what did not close, why it changed, and what it became.

#	First edition	Problem found in review	This edition
1	ϕ maps frames to “object-level state”	Object and state undefined; objects nest and occlude	s is a tree; minimal bounding-rectangle top-left + mask + color map + layer; occlusion completed from history; ϕ_0 is a prior, ϕ_h part of the hypothesis
2	Position as centroid	A centroid cannot redraw the object	Position = top-left of the minimal axis-aligned rectangle in frame coordinates
3	L_fit computed on x as T_h(x, a)	T* is unobservable because of latent z; T_h should act on z	Enc_h estimates $\hat{z}$ from history; T_h($\hat{z}$, a); R_h renders back for comparison
4	L_len “model length”; “an MDL form”	Long $\neq$ complex (long variable names); we model ourselves rather than apply MDL	C(h) model complexity (ideally K(h)); “formally similar to MDL”
5	“h is trustworthy enough”	Undefined	Candidates agree along the planned path; same source as the exploration criterion; ϵ open
6	Whether RESET counts unverified	Check the source	Ordinary RESET counts 1 action; full reset opens a new record
7	Chapters 4/5 split offline vs online	Offline class-B methods are also LLM + prompt and also learn T_h (as text); the real difference was hidden	Classes A/B/C by the form of h
8	Duck’s outer layer “has no learning”	The outer model exists; its learning method is people supplying ideas and prompts	$\Lambda = (\theta, P, \text{tools})$; θ learned by pretraining, P by humans in the loop
9	Duck stores h as code	Source: every Python call starts fresh	h is text in context, evicted with it
10	Duck continues StochasticGoose’s method	The methods are completely different	Lineage only (same lab)
11	Blind Squirrel’s ResNet trained from scratch, same level only	Author’s repo: ImageNet-pretrained, cross-level fine-tuning	Outer visual features + inner cross-level generalization
12	Click map “probability each pixel changes”	Unclear	Probability the whole frame changes if that coordinate is clicked
13	AERA’s 0.30 attributed to its LLM agent	Paper: the competition entry is a BFS pre-solve	Described separately, protocol difference noted
14	Why online beats offline not decomposed	Outer, inner or inference?	Mainly θ , then inference budget; inner representation about +9 points
15	“GPT-5.4 is not necessarily larger”	Too conservative	External estimates in the trillions, about two orders of magnitude larger

Other points: in ordinary RL training and inference see the same signals, whereas public-game source code is privileged information (teacher–student); outer = “which base classes and interactions exist”, inner = “how this game defines them and how to win”; leaderboard climbing is not the goal — a good method makes a good paper.

Sources

- [ARC Prize 2026 Milestone 1 blog](#)
- [ARC-AGI-3 Preview 30-day learnings](#)

- [Duck harness source](#) · [Tufa Labs write-up](#)
- [StochasticGoose source](#)
- [Blind Squirrel source](#)
- [Graph-Based Exploration \(arXiv 2512.24156\)](#)
- [AERA \(arXiv 2605.25931\)](#)
- [Executable World Models \(arXiv 2605.05138\)](#) · [code and run artifacts](#)
- [Do Coding Agents Need Executable World Models... \(arXiv 2607.15439\)](#)
- [Tycho \(arXiv 2607.28287\)](#) · [code](#)
- [Kepler \(arXiv 2610.00834\)](#)
- [OPINE-World \(arXiv 2607.01531\)](#)
- [ARA trajectory example](#)
- [ARC-AGI-3 Technical Report](#)
- [arc-agi 0.9.9 / arcengine source \(PyPI, `scorecard.py`, `models.py`\)](#)